

Session 1 · Deck 1.B3

From Dynamic Programming to Reinforcement Learning — and from One Agent to Many

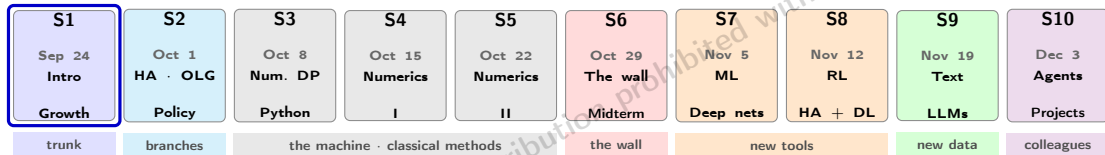
Zhigang Feng

Thursday, September 24, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

▼ you are here: 1.B3



Build it on paper → solve it on the machine → push it to the frontier with AI

This deck: reinforcement learning is dynamic programming with the expectation *sampled* and the value function *learned*; heterogeneous agents turn the RL “environment” into an *equilibrium object*.

Same Equation, Three Worlds

The growth model

2

state variables (k, z)

rules: **known**

value table: **easy**

solved by VFI (1.B2)

The game of Go

$\approx 10^{170}$

legal positions

rules: **known**

value table: **impossible**

AlphaGo: self-play +

neural nets + search

A heterogeneous-agent economy

∞

a distribution in the state

rules: known *individually*

aggregate law: **endogenous**

this course, S5–S8

All three are **Bellman equations**. Go shows that “the model is known” does not mean “the table can be written.” The economy adds a twist Go does not have: **the environment is made of everyone else’s choices**.

Same Problem, Two Languages

MDP	Growth model
$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$	
state s	(k, z)
action a	k' (equivalently c)
transition $P(s' s, a)$	k' chosen; $z' \sim \Pi(\cdot z)$
reward $R(s, a)$	$u(c)$
discount γ	β
policy $g(s)$ (RL: π)	decision rule $k' = g(k, z)$
return $G_t = \sum_j \gamma^j r_{t+j+1}$	lifetime discounted utility
episode	a simulated path

Economist:

$$V(k, z) = \max_{k'} \{u(c) + \beta \mathbb{E}[V(k', z') | z]\}$$

RL researcher:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \left\{ R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right\}$$

Identical recursive skeleton. The two traditions differ in **what they assume is available**: economists write P down; RL assumes only that it can *sample* from it.

Values, Actions, and Why RL Prefers Q

Evaluate a policy g (linear, no max):

$$V^g(s) = R(s, g(s)) + \gamma \sum_{s'} P(s' | s, g(s)) V^g(s')$$

Control (Bellman optimality):

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a')$$

Acting on each:

$$g^*(s) = \arg \max_a \left\{ R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right\}$$

$$g^*(s) = \arg \max_a Q^*(s, a)$$

Why Q ?

To act on V^* you still need the transition P to compare actions. To act on Q^* you only compare numbers in a row of a table. Learning Q moves the expectation *into the data* — so improvement needs no model.

The cost: $|S| \times |\mathcal{A}|$ numbers instead of $|S|$. With continuous actions (portfolio shares, investment) you need a different trick — policy gradients (S8).

A Growth Model, Solved as a Table

$k \setminus k'$	1.50	2.25	3.00	3.75	4.50	5.25	6.00
1.50	0.563	-0.004	—	—	—	—	—
2.25	1.226	1.932	1.441	—	—	—	—
3.00	1.609	2.533	2.946	2.414	—	—	—
3.75	1.879	2.898	3.511	3.735	3.054	—	—
4.50	2.088	3.160	3.864	4.277	4.275	3.620	—
5.25	2.259	3.366	4.122	4.623	4.803	4.784	4.126
6.00	2.403	3.534	4.325	4.877	5.144	5.303	5.257

$Q^*(s, a)$ for $z = 0.9$; "—" = infeasible ($c \leq 0$); bold = row maximum $g^*(k, 0.9)$.

Seven capital grid points, $z \in \{0.9, 1.1\}$ with persistence 0.8, $\beta = 0.95$, $\delta = 0.10$,

$\alpha = 0.36$, $u = \ln c$. Source: Feng & Miao, *AI for Economists*, ch. 12.

- The whole solution of a tiny growth model is a table: 14 states $\times$ 7 actions (the $z = 0.9$ half is shown).
- The optimal policy moves capital toward the steady state (≈ 3.8).
- Row by row, best and second-best can be far apart ($k = 3.75$: 0.224) or nearly tied ($k = 4.50$: 0.002).

Reinforcement learning tries to fill in this table from simulated experience, without ever using the transition probabilities. The near-ties are where it will struggle.

Full Backup vs. Sample Backup

Dynamic programming — full expectation backup, every state:

$$Q_{n+1}(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_n(s')$$

Monte Carlo — simulate N paths from (s, a) and average the returns:

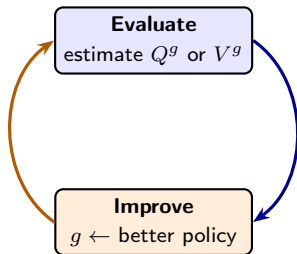
$$\hat{Q}(s, a) = \frac{1}{N} \sum_{i=1}^N G^{(i)}, \quad \text{s.e.} \approx \sigma_G / \sqrt{N}$$

Temporal difference — one sampled step, then your own estimate: $r + \gamma V_n(s')$.

	exact expectation (needs P)	one sample (needs a simulator)
one step	DP / VFI backup	TD(0), Q-learning
whole path	policy evaluation (Howard)	Monte Carlo

In the table model at $(k, z) = (3.75, 0.9)$: $\sigma_G \approx 0.74$. Telling apart two actions whose values differ by the median gap 0.085 with 95% confidence takes **≈ 400 paths per cell**. Error falls with $\sqrt{N}$ — *regardless of the number of states*.

The Common Skeleton: Generalized Policy Iteration



stop when neither step changes

Algorithm	Evaluate by	Improve by
Policy iteration	exact linear solve	greedy arg max
Value iteration	one full backup	greedy arg max
Monte Carlo control	average of sampled returns	ϵ -greedy
Q-learning / TD	one sampled step + own estimate	ϵ -greedy
Actor-critic	a critic network	a policy-gradient step on an actor network

One sentence to remember: reinforcement learning solves the Bellman equation with *sampling* (for the expectation) and *function approximation* (for the value and policy), organized as an evaluate-improve loop you already know from policy iteration.

The Price of Sampling: Noisy Estimates and Exploration

At $(k, z) = (3.75, 0.9)$ the five feasible actions are learned from sampled returns, starting from **one path per action**. After 300 rounds, how often is the chosen action the optimal one?

Rule for new samples	optimal	samples/round
pure greedy	0.66	1.0
random re-check (10%)	0.85	1.0
periodic re-check	0.98	1.4
sample every action	1.00	5.0

2,000 replications. Source: Feng & Miao, *AI for Economists*, ch. 12.

Two different problems

Precision: estimates are noisy — fades as samples accumulate.

Coverage: an action that was unluckily underestimated is never tried again, so it is *never corrected* — does not fade with time.

Hence **exploration:** ϵ -greedy, exploring starts. DP never faces this — its max *inspects* every action instead of trying it.

For economists: a central bank cannot “explore” interest rates for fun (and the Lucas critique bites). That is why we train agents inside **calibrated structural models**.

When Dynamic Programming, When Reinforcement Learning?

	model known	model unknown
low-dim	classical DP: VFI, policy iteration	tabular RL: Q-learning, SARSA
high-dim	"deep DP": neural nets inside VFI or Euler methods (S7)	deep RL: heterogeneous agents, market design (S8)

Structurally, DP is a limit case of RL: with P known, a sample backup becomes a full backup; with few states, function approximation becomes a table.

What matters for macro

In macroeconomics the model is usually *known* — we wrote it. RL's value is not "model-free"; it is **simulation instead of integration** and **learned functions instead of grids** in high dimensions.

But the guarantees thin out

The contraction argument covers tables. Combine function approximation, bootstrapping and off-policy data (the "deadly triad") and convergence is no longer guaranteed. **Verification becomes your job** (1.B2).

From One Agent to Many: The Household's MDP

A household in a heterogeneous-agent economy with aggregate risk:

$$V(a, z; Z, \Gamma) = \max_{a' \geq \underline{a}} \left\{ u(c) + \beta \mathbb{E}[V(a', z'; Z', \Gamma')] \right\}$$

$$c = (1+r(Z, \Gamma))a + w(Z, \Gamma)z - a', \quad \Gamma' = H(\Gamma, Z, Z').$$

state	$(a, z; Z, \Gamma)$ — Γ is a <i>distribution</i>
action	a'
reward	$u(c)$
transition	$z' \sim \Pi_z, Z' \sim \Pi_Z, \Gamma' = H(\Gamma, Z, Z')$

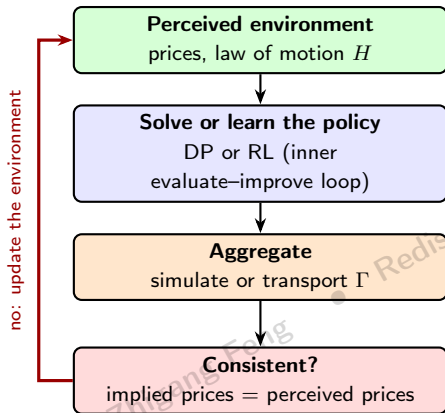
Difficulty 1: dimension

Prices depend on the distribution Γ over assets and income — an infinite-dimensional object. Even a 100-bin histogram with 7 income states is 700 numbers in the state.

Difficulty 2: the law of motion

H is generated by **everyone's policy** g . It has no closed form in general, and it changes whenever g changes.

The Environment Is an Equilibrium Object



- Standard RL treats the environment as **fixed**. In general equilibrium the environment is **made of the policy being learned**.
- A heterogeneous-agent model is **RL plus a consistency loop** — the RCE consistency condition from 1.B1.
- Krusell-Smith runs the outer loop with a few moments; modern methods use neural networks and simulated or exact distributions.

Research in the wild

In our taxation project the government is a second learner whose rule reshapes the households' environment: two nested consistency loops (S8).

How This Course Handles the Distribution

Strategy	What it assumes / does	Examples	Where
Stationary equilibrium	No aggregate risk: λ and prices are constant. Inner DP for households, outer loop on the interest rate.	Huggett, Aiyagari	S2, S5
Approximate aggregation	Agents forecast prices with a few moments of Γ ; iterate on the forecasting rule.	Krusell–Smith (1998)	S2, S6
Linearize around the stationary distribution	Small aggregate shocks; perturbation in the distribution itself.	Reiter (2009); sequence-space Jacobian	S6
The distribution as a network input	Histograms or learned moments feed neural networks; simulate or transport Γ exactly.	DEQN, DeepHAM, actor–critic with an analytic kernel	S8

All four are ways to make “**what must agents forecast, and what makes the forecast correct?**” computable.

The Trunk as a Platform: What Each Extension Adds

Extension	What changes in the recursive problem	Where
Labor–leisure choice (RBC)	one intratemporal condition; the state is unchanged	HW1
Distortionary taxes, frictions	welfare theorems fail: solve the competitive equilibrium directly	S2
Idiosyncratic risk + borrowing limits	individual state (a, z) ; a stationary distribution	S2, S5
Aggregate risk	the distribution enters the state	S2, S6, S8
Finite lives	age enters the state; cohort distributions and transitions	S2, S6
Government without commitment	policy is a rule that must be a best response to itself	S2, S6, S8
Health, job search, entrepreneurship	extra states and discrete choices; non-convexities	projects

Each row is a research literature — and each adds either a **dimension** or a **fixed point**.

Session 1 in One Slide — and Next Week

1. The growth model is the trunk: planner, market, and recursive equilibrium coincide — and the **RCE template** (state, forecast, consistency) organizes every later model.
2. Dynamic programming is trustworthy (equivalence, contraction, properties) and computable (VFI, policy iteration, Euler methods) — **with verification**.
3. Reinforcement learning is the same evaluate–improve loop with **sampled expectations** and **learned functions**.
4. With many agents, the environment is an **equilibrium object**: learning must be wrapped in a consistency loop.

Next week (S2): why can't everyone insure?

Bewley–Huggett–Aiyagari, Krusell–Smith, overlapping generations, and optimal policy with and without commitment.

Exit ticket before you leave: one thing that clicked today, one thing that did not.

Before then: Ljungqvist–Sargent ch. 3–4 or

Stokey–Lucas–Prescott ch. 4; Sutton–Barto ch. 3–4 (optional); install Python with the setup lab.